

The University of Hong Kong

Faculty of Science

Master of Science in Artificial Intelligence Programme

ARIN7600 Artificial Intelligence Project 2026/9

Interim Report

**Topic: BNPDFormer——Dynamic bottleneck prediction in digital
manufacturing**

Student: Tian Yuxiao(3036609589), Xu Wantong(3036580161), Zhang Haowen(3036580367)

Supervisor: Prof. Xiao Li

Submission Date: September 6, 2026

Contents

Abstract	3
1 Introduction	4
1.1 Project Background	4
1.2 Project Overview	5
1.3 Project Objective and Current Contributions	5
1.4 Outline of the Progress Report	6
2 Related Work	7
2.1 Throughput Bottleneck Identification and Prediction	7
2.2 Spatiotemporal Forecasting on Graphs	8
2.3 Digital-Twin-Oriented Simulation for HRC Factories	9
2.4 Research Synthesis and Project Positioning	10
3 Methodology	11
3.1 Simulation Data Engine	11
3.1.1 Overview	12
3.1.2 Isaac Sim Factory Environment	12
3.1.3 Event-Driven Bottleneck Data Collection	13
3.1.4 Disturbance Dimensions and Injection Protocol	13
3.1.5 Offline Feature Aggregation and Labeling	14
3.1.6 Streaming Aggregation and Online Inference	14
3.2 BNPDFormer Architecture	15
3.2.1 Problem Formulation and Schema	15
3.2.2 Spatiotemporal Encoder	16
3.2.3 Dual-Path Prediction Heads	17
4 Current Progress	18
4.1 Digital Twin Factory Environment	18
4.2 Dataset Construction and Schema Instantiation	19
4.3 Model Implementation and Preliminary Experiments	20
4.4 Challenges	22
5 Conclusion	24

6	Future Plans	25
6.1	Track I: Systematic Evaluation and Model Refinement	25
6.2	Track II: Product-Side Forecasting and Shop-Floor Deployment	25
	References	28

Abstract

Throughput bottlenecks on human–robot collaborative (HRC) factory lines are dynamic, multi-resource, and sparse in time. Operators need precision-first warnings of *which* station will congest, *when* congestion starts, and *how long* it lasts, rather than retrospective detection alone.

This project builds a labelled digital-twin data engine in NVIDIA Isaac Sim (`isaac_factory`) and a spatiotemporal forecasting model (**BNPDFormer**) that emits station-level bottleneck events on a heterogeneous factory graph. We reconstruct a modular-construction water-pipe production-part line (storage, gantry/AGV logistics, cutting/grooving/welding/painting, workbenches) and collect event-driven raw logs under controlled single-axis disturbances (machine, human, logistics, material). Offline aggregation produces windowed operational features, occupancy labels, and sparse bottleneck events.

BNPDFormer encodes these features with a PDFormer-style backbone (temporal, geographic, and delay-aware semantic attention) and predicts, over a fixed short horizon, a binary will-block flag together with start minute and duration. Auxiliary heads regress remaining order completion time and a coarse process-cause class.

On 134 usable short-order episodes, BNPDFormer attains test report precision/recall/F1 of 0.817/0.447/0.578 with start MAE 0.20 min, showing that precision-first station warnings are feasible; upcoming-onset recall remains the open problem. Overall, the current progress of this project has established (i) a reproducible Isaac Sim collection–aggregation pipeline and (ii) a BNPDFormer prototype capable of station-level bottleneck event forecasting, laying the foundation for systematic evaluation and shop-floor-oriented deployment.

Chapter 1

Introduction

1.1 Project Background

Modular construction transfers many construction activities from building sites to controlled off-site factory environments, creating opportunities for greater automation and more consistent production. These factories nevertheless depend on machine processing, manual work, material handling, and human–robot collaboration (HRC). This combination reflects the human-centred vision of Industry 5.0: machines and robots provide repeatability and physical capacity, while workers contribute adaptability in variable or less structured tasks [1, 2]. Although each product follows a defined processing route, the factory as a whole is not a simple serial line. Its throughput depends on interactions among stations, workers, gantries, automated guided vehicles (AGVs), buffers, and material availability.

Within such an interconnected factory, a throughput bottleneck is the resource that places the greatest constraint on overall production performance; it is not necessarily the one with the longest queue or highest utilization [3]. This constraint can shift as jobs progress and as machine failures, worker availability, transport delays, or material shortages alter the balance of the system. A local disruption may create upstream accumulation and downstream starvation after a delay; as these effects propagate, the limiting resource may change over the course of an order rather than remain fixed.

Despite this dynamic behaviour, established methods mainly identify current or historical bottlenecks from queue states, process states, or combined operational signals [3, 4, 5]. Although valuable for diagnosis, retrospective detection may leave insufficient time for preventive action. A more actionable forecast should indicate whether a bottleneck will occur, where it will occur, when it will begin, and how long it will last. This motivates an event-oriented forecasting problem rather than only future-utilization prediction or bottleneck-node classification. Reliable warnings are particularly important because false alarms may trigger unnecessary rescheduling or intervention.

Developing such forecasts, however, requires suitable labelled data. Obtaining sufficiently varied labels from a real factory is difficult because deliberately inducing failures or shortages would disrupt production, while naturally occurring events and relevant internal states may be sparse or unavailable at the required temporal resolution. A digital-twin-oriented simulator offers controlled and repeatable observations of resource states, event times, and operational responses [6]. Unlike real-world disruptions, simulated disturbances can be repeated under known schedules

while preserving the production controller. This project therefore uses NVIDIA Isaac Sim and Isaac Lab to reconstruct a water-pipe component production line for modular construction and generate quality-gated trajectories under controlled machine, human, logistics, and material disturbances. Building on this data engine, **BNPDFormer** forecasts station-level bottleneck events from recent operations on the heterogeneous factory graph. In this report, *station-level* is used operationally to include all reportable production and logistics resources—processing stations, gantries, and AGVs—rather than processing stations alone.

1.2 Project Overview

The project is organized as two connected phases.

Phase I — simulation data engine and operational event labels. The first phase develops a reproducible Isaac Sim data engine for the water-pipe production line and records production and logistics operations under controlled disturbances. Bottleneck labels are derived from sustained combinations of observable evidence, including queue build-up, blockage, starvation, transport delay, material-shortage propagation, and resource unavailability. The presence of an injected disturbance alone does not create a bottleneck label.

Phase II — spatiotemporal forecasting and controlled comparison. The second phase represents production and supporting resources as a heterogeneous factory graph and develops graph-based spatiotemporal forecasting for upcoming station-level bottleneck events. A shared protocol has been established to compare BNPDFormer with XGBoost, LSTM, a BTGCN adaptation, and a BSTAN-style baseline, covering tabular, sequential, and graph-based alternatives.

Together, the two phases provide a traceable workflow from controlled simulation runs to comparable station-level bottleneck event forecasts.

1.3 Project Objective and Current Contributions

The project objective is to develop and evaluate a simulation-grounded method for forecasting station-level bottleneck events from recent operational histories on a heterogeneous HRC factory graph. Each forecast should indicate event occurrence and, when an event is reported, specify the affected reportable resource, onset, and duration using labels grounded in observable factory behaviour.

The study addresses three research questions:

RQ1: Can recent operational histories of heterogeneous factory resources be used to predict whether an upcoming station-level bottleneck event will occur and which reportable re-

source it will affect?

RQ2: Does modelling process–logistics topology improve forecasting relative to tabular and sequential baselines, and does propagation-delay-aware modelling add value beyond graph-based baselines?

RQ3: Can the predicted bottleneck events achieve sufficiently accurate onset and duration estimates to support actionable early warning?

At the interim stage, the project makes the following contributions:

- **A controllable HRC factory data engine.** The simulated water-pipe line supports reproducible disturbances across machines, humans, logistics, and materials and produces quality-gated operational trajectories for forecasting research.
- **An operational labelling framework.** The framework derives persistent station-level bottleneck event labels from queue, resource-state, transport-delay, material-shortage, and propagation evidence, while treating injected disturbances as contextual causes rather than labels.
- **An event-oriented graph forecasting model.** BNPDFormer adapts propagation-delay-aware spatiotemporal modelling to heterogeneous factory operations and estimates event occurrence and, for reported events, the affected reportable resource, onset, and duration.
- **A shared evaluation protocol.** An episode-level, leakage-controlled protocol aligns inputs, targets, splits, and report-level evaluation across BNPDFormer and four baselines, enabling consistent comparison of resource localization and event timing. The formal comparison remains in progress.

1.4 Outline of the Progress Report

The remainder of this report is organized as follows. Chapter 2 reviews bottleneck research, graph forecasting, and digital twins for HRC manufacturing. Chapter 3 presents the methodology, while Chapter 4 reports current progress and challenges. Chapters 5 and 6 summarize the interim findings and planned work.

Chapter 2

Related Work

2.1 Throughput Bottleneck Identification and Prediction

Throughput bottlenecks are generally understood as the stations or resources with the greatest limiting effect on overall production performance. A systematic review organizes manufacturing bottleneck-detection methods according to the operational information they use: queue-state information, process-state information, or a combination of both [3]. Queue-state approaches characterize accumulated work and waiting behaviour, whereas process-state approaches examine how resources transition among operating states. Combined methods integrate these complementary signals to better distinguish a true throughput constraint from symptoms propagated elsewhere in the line. This distinction matters because neither a long queue nor high utilization alone proves that a resource limits system throughput.

Queue-state methods form one important line of development. They identify potential constraints from queue length or the time that jobs wait before processing [3]. These measures reveal where work accumulates and are comparatively easy to interpret, but a long queue can be an upstream symptom of a downstream constraint rather than direct evidence that the adjacent resource limits system throughput.

The process-state stream has earlier roots and focuses on how resources spend their time. In 2001, Roser, Nakano, and Tanaka defined an active period as an uninterrupted interval during which a resource remains active rather than waiting or being blocked. The resource with the longest average active period is identified as the bottleneck [4]. They extended this idea in 2002 through the shifting-bottleneck method, which uses overlapping active periods to distinguish momentary, sole, and shifting bottlenecks [7].

A related combined-state stream integrates process states with buffer or propagation evidence. In 2009, Li, Chang, and Ni proposed a data-driven turning-point method that uses blockage and starvation probabilities together with buffer records for short- and long-term detection [5]. In 2022, Su et al. combined effective-buffer decomposition, fine-grained resource states, and turning-point theory to identify dynamic bottlenecks in complex coupled systems [8]. Together, these studies support definitions that combine queue, process-state, and propagation evidence rather than relying on a single utilization measure.

Beyond detection, the literature has also moved towards proactive prediction. In 2018, Subramaniyan et al. forecast future active periods with ARIMA before applying active-period

bottleneck detection to automotive production data [9]. More recent learning-based approaches capture nonlinear temporal dependencies; in 2023, Ma et al. proposed an integrated spatial–temporal neural network for high-variety shops with changing relations among workstations [10]. These studies demonstrate that future bottlenecks contain learnable temporal and structural patterns. Across the manufacturing studies reviewed here, however, outputs usually focus on bottleneck identity or a future machine state. A complete station-level bottleneck event formulation that jointly evaluates occurrence, the affected reportable resource, onset, and duration remains uncommon.

2.2 Spatiotemporal Forecasting on Graphs

A manufacturing system can be represented as a graph whose nodes denote tasks, stations, workers, transport resources, or buffers, while edges describe process and logistics relations. This representation makes it possible to model how local resource states evolve over time and affect other parts of the system.

Manufacturing-specific studies demonstrate the value of combining spatial structure with temporal dynamics. In 2023, Lai et al. proposed the Bottleneck Spatial–Temporal Attention Network (BSTAN) [11]. BSTAN represents a production line as a weighted graph, uses graph attention to learn unequal interactions among connected stations, and applies gated recurrent units to forecast future blockage and starvation before identifying bottleneck locations. Its direct focus on manufacturing stations and future bottleneck indicators makes it closely aligned with the present task.

In 2025, Yin, Ma, and Ge proposed the Bottleneck Spatio-Temporal Graph Convolutional Network (BTGCN) for task-bottleneck prediction in digital manufacturing enterprises [12]. BTGCN constructs a task network and combines graph convolution with gated recurrent units to capture structural dependencies among tasks and temporal changes in task execution. It provides a graph-sequential formulation, although its original task-network setting is less directly aligned with station-level blockage and starvation in a production line.

The project therefore uses a BSTAN-style baseline as the closest manufacturing-specific graph baseline for the present task, together with a BTGCN adaptation as a complementary graph baseline. Neither baseline is an exact reproduction of the original model; both are adapted to the shared station-level bottleneck event task. Their relative performance will be established through the formal controlled comparison.

Traffic forecasting offers additional methods for modelling dependencies that are not captured by local adjacency alone. PDFormer introduces explicit geographic and semantic attention masks for short- and long-range relations, together with a transformation designed to represent propagation delay [13]. These ideas are relevant to factories because a disruption can influence non-

adjacent resources through delayed production and logistics effects. PDFormer therefore motivates the examination of whether topology, behavioural similarity, and propagation-delay awareness provide information beyond the local graph-sequential structures represented by BSTAN and BTGCN.

Dense state forecasting must also be distinguished from sparse event timing. STGNPP combines a spatiotemporal graph encoder with a neural point process to predict the occurrence time and duration of sparse traffic-congestion events [14]. Its formulation motivates treating onset and duration as event properties rather than only forecasting a sequence of dense states, although the present project focuses on reporting all relevant station-level bottleneck events within a fixed future horizon.

The transfer from traffic to manufacturing is non-trivial. Traffic nodes usually have comparable measurement semantics and strong calendar periodicity, whereas factory graphs contain heterogeneous machines, workbenches, gantries, AGVs, workers, and buffers. Production dynamics are also driven by job progress, routing, and resource interactions rather than primarily by time-of-day or day-of-week cycles. Traffic architectures must therefore be adapted to factory-specific node types, process–logistics topology, and progress-dependent behaviour.

2.3 Digital-Twin-Oriented Simulation for HRC Factories

A full digital twin couples a physical system with a synchronized virtual representation through data exchange, supporting monitoring, analysis, prediction, and operational decision-making. Lu et al. emphasize that this connection requires an information model, data-processing capability, and bidirectional communication [6]. For bottleneck research, a simulation model developed within such a workflow can provide complete operational traces and repeatable scenarios that would be difficult or disruptive to collect on a real production line.

The human component remains important in this setting. Industry 5.0 emphasizes human-centric, sustainable, and resilient production [1]. In modular construction manufacturing, HRC combines robotic consistency with human adaptability, but Fu, Chen, and Lu report that adoption remains limited and that human and robot roles vary substantially across tasks [2]. These characteristics motivate representing workers and transport agents alongside processing resources, even when forecasts are reported for production or logistics resources.

The present Isaac Sim environment is therefore more accurately described as a digital-twin-oriented simulation and data engine than as a fully synchronized digital twin. Simulation also does not remove the sim-to-real gap: state-estimation errors, differences in operating policy, unmodelled human behaviour, and imperfect calibration can all affect transfer to a physical factory. Later validation will require mapping the simulated operational signals to MES, PLC, or sensor data and calibrating the environment against physical-line behaviour.

2.4 Research Synthesis and Project Positioning

Task formulation. Existing bottleneck research provides established queue-state, process-state, and combined-state indicators, together with methods for detecting or predicting bottleneck identities. Existing methods less commonly formulate prediction as a complete future event comprising occurrence, the affected reportable resource, onset, and duration. Building on this foundation, the present study evaluates these elements explicitly within a station-level bottleneck event task.

Model positioning. BSTAN and BTGCN combine manufacturing graphs with temporal modelling, while PDFormer contributes explicit long-range and delay-aware mechanisms in traffic forecasting. Taken together, these approaches leave open how such capabilities should be combined for heterogeneous factory resources and progress-driven production dynamics. The project investigates this adaptation without assuming that a traffic model transfers directly to manufacturing.

Data foundation. In physical factories, uncommon disturbances are difficult to repeat under controlled conditions, and complete operational event labels are difficult to obtain. Digital-twin-oriented simulation provides controlled and repeatable trajectories for these events. The project uses this capability to construct process-consistent labels across machine, human, logistics, and material disturbances, while retaining the sim-to-real considerations discussed above.

Taken together, the project is positioned at the intersection of operational bottleneck analysis, graph-based spatiotemporal forecasting, and simulation-grounded HRC manufacturing research.

Chapter 3

Methodology

The project aims to forecast station-level bottleneck events from an Isaac Sim HRC factory twin [6, 2]. The overall framework is shown in Figure 3.1. The methodology consists of a simulation data engine (Section 3.1) and the BNPDFormer model (Section 3.2).

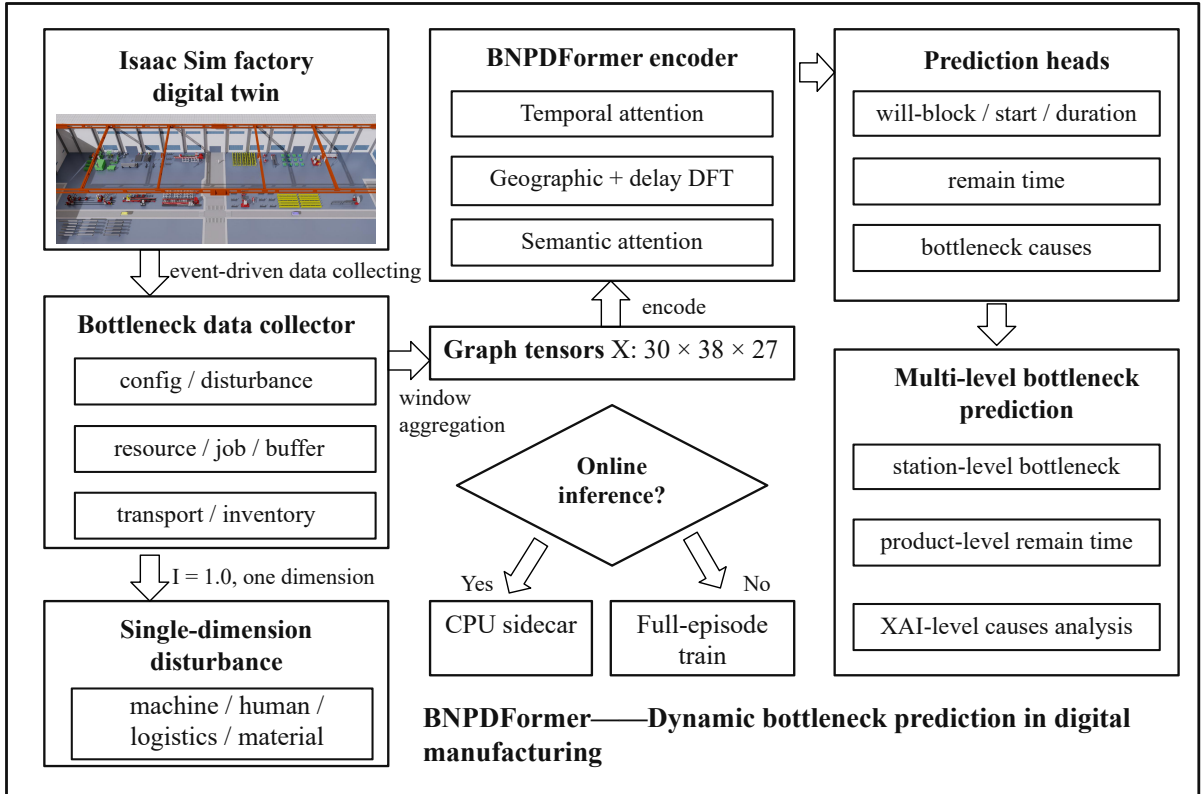


Figure 3.1: Overall methodology framework. Isaac Sim logs are aggregated into graph tensors, encoded by BNPDFormer, and decoded as multi-level bottleneck predictions (station events, remaining time, and cause analysis). A station report is counted correct when the station matches and the start error is at most 3 minutes.

3.1 Simulation Data Engine

This section presents an end-to-end methodology for turning Isaac Sim rollouts into supervised tensors for bottleneck event forecasting. The pipeline integrates a process-faithful factory environment, event-driven raw logging, controlled disturbances, offline window aggregation, and an optional streaming sidecar for online inference.

3.1.1 Overview

Given one simulation episode, the pipeline executes as follows:

1. Environment rollout: rule-based control of process and logistic tasks on the water-pipe line.
2. Event-driven raw logging of seven tables (config, disturbances, resources, jobs, buffers, transport, inventory).
3. Optional single-axis disturbance injection (machine / human / logistics / material).
4. Offline aggregation into per-resource time windows, occupancy masks, bottleneck events, and job KPIs.
5. Export to graph tensors and BNPDFormer training / inference.

3.1.2 Isaac Sim Factory Environment

We reconstruct a water-pipe production-part line in NVIDIA Isaac Sim: storage, gantry/AGV logistics, cutting/grooving/welding/painting, and workbenches. A rule-based controller executes process and logistic tasks. Each training episode orders 10 pipes with WIP cap 5.

The simulated line follows a process–logistics topology in which raw pipes are released from storage, transported by gantry/AGV assets, processed at cutting, grooving, welding, and painting stations, and then transferred to downstream workbenches or the next machine according to process precedence. The environment maintains multiple resource types, including machines, workbenches, transport assets, gantries, human operators, buffers, and material inventories. These heterogeneous resources are unified in a common state dictionary so that scheduling, logging, and later graph construction operate on the same identifiers and state transitions.

Time advancement is split into logic time and physics time. The logic layer updates task allocation, product progress, and resource states once per decision step, while the physics layer executes motions and interactions inside Isaac Sim. This separation keeps the production semantics explicit and also allows the collector to record process-faithful events at stable logical timestamps.

To stabilize long rollouts, the environment applies basic production safeguards. A work-in-process cap prevents uncontrolled queue growth, admission control limits how many products can be active simultaneously, and deadlock-related checks are used to avoid circular waiting among machines, logistics assets, and human resources. Together, these safeguards improve data quality and ensure that bottleneck events arise from controlled operational pressure rather than simulator breakdown.

3.1.3 Event-Driven Bottleneck Data Collection

A read-only collector attached at the end of the environment step writes seven raw tables without modifying the task manager, agents, or physics. Time is recorded as logical step and logic seconds; resource IDs are stable within a run. Table 3.1 summarizes the schema.

Table 3.1: Core raw log tables written by the Isaac Sim collector.

Table	Role
episode_config.csv	Run / episode metadata, disturbance dim & intensity
disturbance_log.csv	Injected shocks (L0/L1/L2) as environment context
resource_event_log.jsonl	Resource state transitions
job_trace.csv	Job / subtask boundaries
buffer_event_log.csv	Buffer occupancy events
route_transport_task.csv	Gantry / AGV transport tasks
material_inventory_log.csv	Kit / inventory snapshots

We deliberately adopt event-driven logging rather than periodic full-state dumps. Periodic dumps are simple but produce redundant snapshots, blur state transitions, and create unnecessary I/O overhead for long episodes. In contrast, the collector writes a new record only when a resource state, job stage, buffer occupancy, transport task, or inventory status changes. This design preserves exact boundaries of operational events, keeps the logs compact, and is better aligned with later window aggregation.

The collector is versioned in the output metadata so that downstream aggregation can remain reproducible even if the schema evolves. Raw logs are organized hierarchically as `output/bottleneck_dataset/<run_id>/episode_XX/env_YY/`, while derived window tables and labels are stored under the corresponding `derived/` directory.

3.1.4 Disturbance Dimensions and Injection Protocol

Formal training uses intensity $I = 1.0$ and *one* primary dimension per run: machine, human, logistics, or material (plus scarce no-disturbance controls). Configuration changes (fewer workers/AGVs, closed stations) combine with episodic shocks (faults, leave events, freezes, temporary kit shortages). Disturbances are *environment context*, not bottleneck labels: L2 intervals enter features as `disturbance_active_s` but are not treated as occupancy events.

Disturbance intensity is defined as a ladder that controls how strongly the factory deviates from its nominal operating condition. In the current training setup, $I = 1.0$ is the standard level and is used to generate the main in-distribution dataset. At this level, each run activates exactly

one primary disturbance dimension. For machine disturbances, the simulator applies moderate workstation unavailability, processing slowdowns, or short fault intervals; for human disturbances, it reduces effective labour capacity through fewer available workers or slower manual execution; for logistics disturbances, it lowers transport capacity by reducing AGV availability, slowing gantry/AGV service, or inserting temporary freezes; for material disturbances, it injects bounded supply delays or temporary kit shortages.

Higher intensity levels such as $I = 2.0$ or $I = 3.0$ are reserved for stress testing and ablation analysis, where the same disturbance dimension is amplified to create more severe bottleneck propagation. Mixed-dimension disturbances are not used in the main training set; instead, they are reserved for out-of-distribution evaluation to examine whether the model trained on clean single-axis supervision can generalize to more realistic compound disruption scenarios.

3.1.5 Offline Feature Aggregation and Labeling

Heterogeneous raw states are aggregated into per-resource windows of $\Delta = 60$ s. Unsupervised operational occupancy (turning-point, queue buildup, material starvation, inbound delay, upstream blockage, score threshold) yields dense occupancy grids and sparse `bottleneck_event` runs [5, 4, 8]. Job KPIs supply remaining-order length. Supervision-only scores are excluded from input features to avoid label leakage.

For each prediction point, the model consumes a history of $T_{\text{in}} = 30$ windows, corresponding to the previous 30 minutes of system evolution, and predicts events over a short horizon of $H = 15$ minutes. This design matches the intended use case of shop-floor warning, where operators require a moderate look-back for context and a short but actionable look-ahead for intervention.

A window on a station node is marked hot when process-consistent indicators jointly suggest operational congestion, such as queue build-up, prolonged waiting, repeated downstream blockage, upstream starvation propagation, inbound logistics delay, or a bottleneck score exceeding a preset threshold. Consecutive hot windows are then merged into occupancy runs and further compressed into sparse bottleneck events with station identity, start time, and duration. To prevent leakage across sequences, training, validation, and test data are split at the episode level rather than by randomly shuffling windows, so that all windows from the same rollout remain in the same partition.

3.1.6 Streaming Aggregation and Online Inference

Isaac Sim writes only raw tables. A CPU sidecar can close finished 60 s windows into `derived/` while collection continues; a trained checkpoint then emits station occupancy / events. Training still requires a full-episode aggregation, because future labels such as `will-block` need a look-

ahead.

For online use, the aggregation sidecar distinguishes between open and closed windows. Only closed windows, whose full 60-second contents have already been observed, are eligible to enter the derived tables and the inference buffer. A follow-mode flag allows the sidecar to keep tracking the newest raw records, while a closed-window flag determines when a window can be finalized and pushed to the predictor. This mechanism avoids partial-window inconsistency and keeps online features aligned with the offline training definition.

The runtime device split follows the simulation workload: Isaac Sim and rendering remain on the GPU, while log monitoring, window closure, and feature aggregation are executed by a CPU sidecar. At inference time the model is causal: it uses only completed historical windows, current environment context, and previously observed disturbances, and it does not access future occupancy labels, future event boundaries, or any post-hoc information available only after the episode ends.

3.2 BNPDFormer Architecture

To forecast station-level bottleneck events [9, 10], we define a factory graph schema, a PDFormer-style spatiotemporal encoder [13], and dual prediction paths (dense occupancy / event reports, plus sparse timing [14]). This section outlines the problem formulation, the encoder, and the heads.

3.2.1 Problem Formulation and Schema

Let $\mathcal{R} = \{r_1, \dots, r_N\}$ be heterogeneous resources. A process-logistics graph $G = (\mathcal{R}, E)$ encodes upstream/downstream process links, buffer affinity, and logistics attachments [11]. At window t , each resource n has a feature vector $x_{t,n} \in \mathbb{R}^F$. History $X_t \in \mathbb{R}^{T_{\text{in}} \times N \times F}$ is encoded; supervised stations predict

$$(w_{t,n}, s_{t,n}, d_{t,n})$$

over the next H minutes (will-block, start offset, duration). A prediction counts as a correct *report* only if the station matches and $|\hat{s} - s| \leq 3$ min. Table 3.2 lists output heads.

Supervision is applied primarily to station-like production nodes, i.e., machines and workbenches whose future bottleneck events are operationally meaningful to report. Human, transport, gantry, buffer, and inventory-related nodes are retained in the graph as context-only nodes: they are not asked to emit bottleneck reports directly, but they provide explanatory signals about resource competition, upstream starvation, downstream blockage, and logistics delay. This supervised-versus-context separation aligns the learning target with actual decision needs while

Table 3.2: BNPDFormer output heads and their roles.

Head	Form	Role
Will-block / start / duration	per station, horizon H	Primary event report (A.1)
Occupancy grid	$(H \times N)$ score / 0–1	Alignment auxiliary
Remaining order time	scalar windows	Order-level context
Cause class	multi-class	Process reason (A.3)
STGNPP τ, d	sparse next-event	Optional timing path

preserving the causal structure of the production system.

The graph contains typed edges that reflect how congestion propagates through the factory. These include process-precedence edges between upstream and downstream stations, attachment edges between stations and their associated buffers, transport-service edges that connect stations to AGVs or gantries, and service-affinity edges that connect manual workstations to human resources. During message passing, these typed neighbourhoods can be represented either explicitly or through corresponding geographic masks so that the encoder attends to process-relevant neighbours rather than to all nodes uniformly.

3.2.2 Spatiotemporal Encoder

Grouped token embeddings with type one-hots feed stacked spatiotemporal blocks: temporal self-attention per node, geographic attention over process–logistics neighbours with delay patterns, and semantic attention over behaviourally similar nodes [13]. Calendar day/week embeddings from traffic PDFormer are removed. Laplacian positional encodings are derived from the factory adjacency.

The encoder stacks multiple spatiotemporal blocks, each operating on a shared hidden representation. In the current prototype, the exact number of layers and hidden width are treated as tunable implementation hyperparameters and are selected on the validation set, while the architectural principle remains fixed: temporal attention models each node’s history, geographic attention is constrained by the factory adjacency or typed neighbourhood masks, and semantic attention links behaviourally similar nodes even if they are not physically adjacent. This decomposition lets the model capture both direct process interactions and non-local pattern similarity.

Geo and semantic attention are implemented with different masks. The geographic mask preserves only process–logistics neighbours, whereas the semantic mask is built from feature similarity or delay-pattern similarity over recent windows. If delay motifs are incorporated, they are used to express recurring lag structures such as “upstream blockage now, downstream starvation several windows later”, which are common in manufacturing propagation chains. These delay-aware patterns complement the static graph structure and improve the encoder’s sensitivity to temporally shifted causal effects.

3.2.3 Dual-Path Prediction Heads

Path I (dense) predicts occupancy over a fixed $H = 15$ min horizon and converts connected hot segments into (start, duration, station) reports. Path II (sparse, STGNPP-style) models the next process hotspot’s waiting time and duration, censored at remaining order completion [14]. Auxiliary heads regress remaining order time and classify a coarse process cause (not the injected L2 type name).

Training uses a multi-task objective. Binary cross-entropy is used for will-block or occupancy classification, masked regression losses are used for start offset, duration, and remaining-order time, and cross-entropy is used for the coarse cause class. When the sparse timing path is enabled, its waiting-time and duration terms are added as an auxiliary event loss. The relative loss weights are tuned to keep the primary station-event task dominant while allowing the auxiliaries to regularize the shared encoder rather than overwhelm it.

Model selection is based on validation report F1 with a precision preference. When two checkpoints achieve similar F1, the one with higher report precision is preferred because false alarms are particularly costly in shop-floor warning.

The dense occupancy path and the sparse point-process path are intentionally not collapsed into a single head because they solve related but different problems. The occupancy path provides a window-wise description of how congestion evolves across the horizon and is useful for spatial alignment and calibration. The sparse path, in contrast, focuses on the next event’s onset and duration, which is a low-frequency and heavily censored prediction problem. Keeping them separate makes the supervision cleaner, preserves interpretability, and allows the dense path to support event extraction while the sparse path specializes in timing accuracy.

Chapter 4

Current Progress

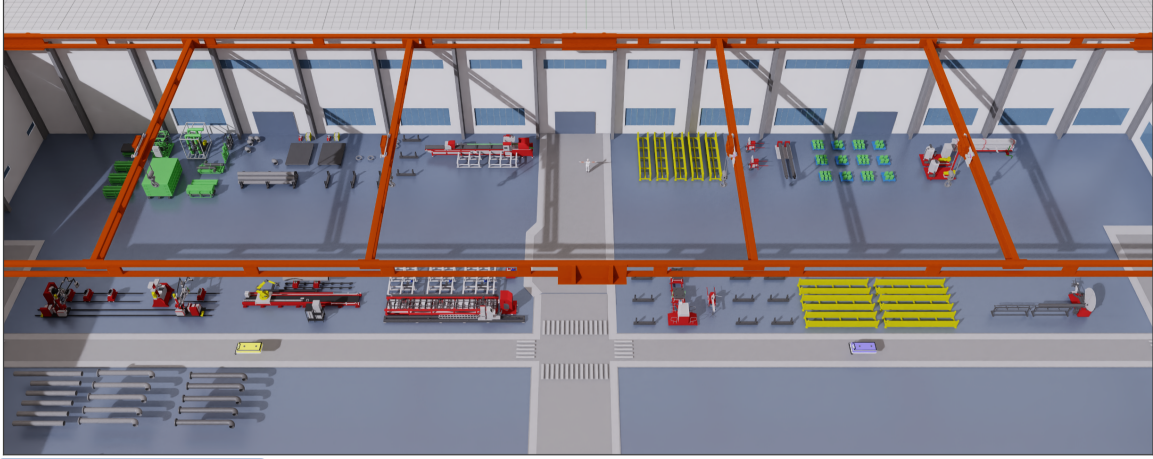
In this chapter, the project progress, up to the submission of this report, is discussed.

Phase I (the Isaac Sim data engine) is operational: `collector v0.3` writes seven raw tables; `bn_agg` derives windows, occupancy, events, and job KPIs; `export` skips deadlock and incomplete episodes. Phase II has a working `BNPDFormer` implementation under `PDFormer/factory_bn/` and a frozen evaluation on the 134-episode usable pack `n10_i1_all_usable`.

4.1 Digital Twin Factory Environment

The Isaac Sim digital twin is the physical–logical substrate of the whole pipeline [6]: it hosts heterogeneous resources, executes rule-based process and logistics tasks, and emits the event logs used for bottleneck labelling. Figure 4.1 summarizes the current twin from two complementary views.

(a) Factory overview



(b) Multi-camera views

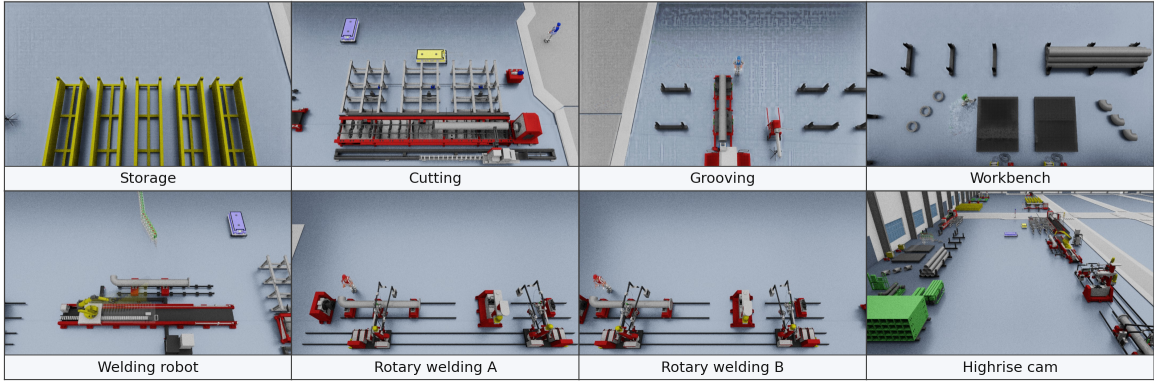


Figure 4.1: Isaac Factory digital twin: (a) factory overview; (b) multi-camera views of key stations.

Panel (a) shows the reconstructed water-pipe HRC line under NVIDIA Isaac Sim, including overhead gantries, storage, processing cells, and workbenches on a shared shop floor. Panel (b) collects station-centric camera streams (storage, cutting, grooving, workbench, welding cells, and a highrise overview), which support visual inspection of resource states during collection and debugging.

This twin-level realization is what later sections instantiate as the seven-table collector, the 38-node factory graph, and the supervised station-event dataset.

4.2 Dataset Construction and Schema Instantiation

The logging schema is instantiated in `isaac_factory` at `src/bottleneck_data.py` (`collector_version=v0.3`). Each short-order episode places 10 water pipes with a WIP cap of 5, under `rule_based` control, at disturbance intensity $I = 1.0$ and a single primary dimension per run. Watchdog false-kills (stall fingerprint ignoring in-transit motion) and WIP-slot bugs that truncated earlier 18-pipe runs have been patched in `hc_single_env_base.py`,

`task_progress_manager.py`, and gantry parking logic.

Export still excludes any episode with `deadlock_reset` or fewer than 10/10 completions. On the current top-level short-order inventory this drops 8 deadlock episodes (mainly material and logistics). The canonical training export `n10_i1_all_usable` contains 134 complete $I = 1.0$ episodes, split at the episode level (92/17/25 train/val/test; seed 42), stratified by run so adjacent windows never leak across partitions.

Table 4.1: Usable $I = 1.0$ episodes in `n10_i1_all_usable` (after deadlock / completeness filters).

Dimension		Runs	Usable episodes
machine	<code>n10_machine1.0 + extra_machine</code>		$20 + 10 = 30$
human	<code>n10_human1.0 + extra_human</code>		$20 + 10 = 30$
logistics	<code>n10_logistics1.0 + extra_logistics</code>		$20 + 10 = 30$
material	<code>n10_material1.0 + extra + recollect</code>		$17 + 9 + 18 = 44$
Total		nine run tags	134

Fifty-five mixed-dimension episodes (`n10_mix_ood`) are held out for out-of-distribution evaluation and are not merged into training.

The factory graph has $N = 38$ nodes: 7 machines/workbenches, 4 gantries, 5 humans, 4 AGVs (`transport_robot`), and 18 buffers/storage cells (Figure 4.2). Each node has $F = 27$ input features: 21 operational columns from `FEATURE_COLS` (queue, wait, occupancy, blockage/starvation, logistics delay, material shortage, disturbance-active context, turning-point flags), 5 resource-type one-hots, and a machine-only `labor_saturated` bit. The supervision score `bottleneck_score_s` is never concatenated into X , to avoid label leakage.

4.3 Model Implementation and Preliminary Experiments

BNPDFormer lives in `PDFormer/factory_bn/`. Training is `python -m factory_bn.train`; inference is `python -m factory_bn.infer` (or `BNPPredictor.predict_x`). The frozen protocol in `FactoryBN.json` uses 60s windows, history $T_{\text{in}} = 30$ (30 min), occupancy horizon $H = 15$ (15 min), encoder depth 4 and embedding width 64, and `use_stgnpp=false`. Connected occupancy cells with `hot_prob` ≥ 0.5 are decoded by `occupancy_to_events` into (start, duration, station) reports; remaining-order length is a separate head.

An older checkpoint `evt/BNPDFormer_best.pt` (WandB `1chf07o9`, test hot F1 ≈ 0.165 , precision ≈ 0.10) used a different occupancy definition and $N = 34$ nodes; its numbers are not comparable to the short-order $H = 15$ protocol and must not be mixed into Table 4.2. On the 134-episode pack, checkpointing maximises validation report F1 with a precision floor. Test

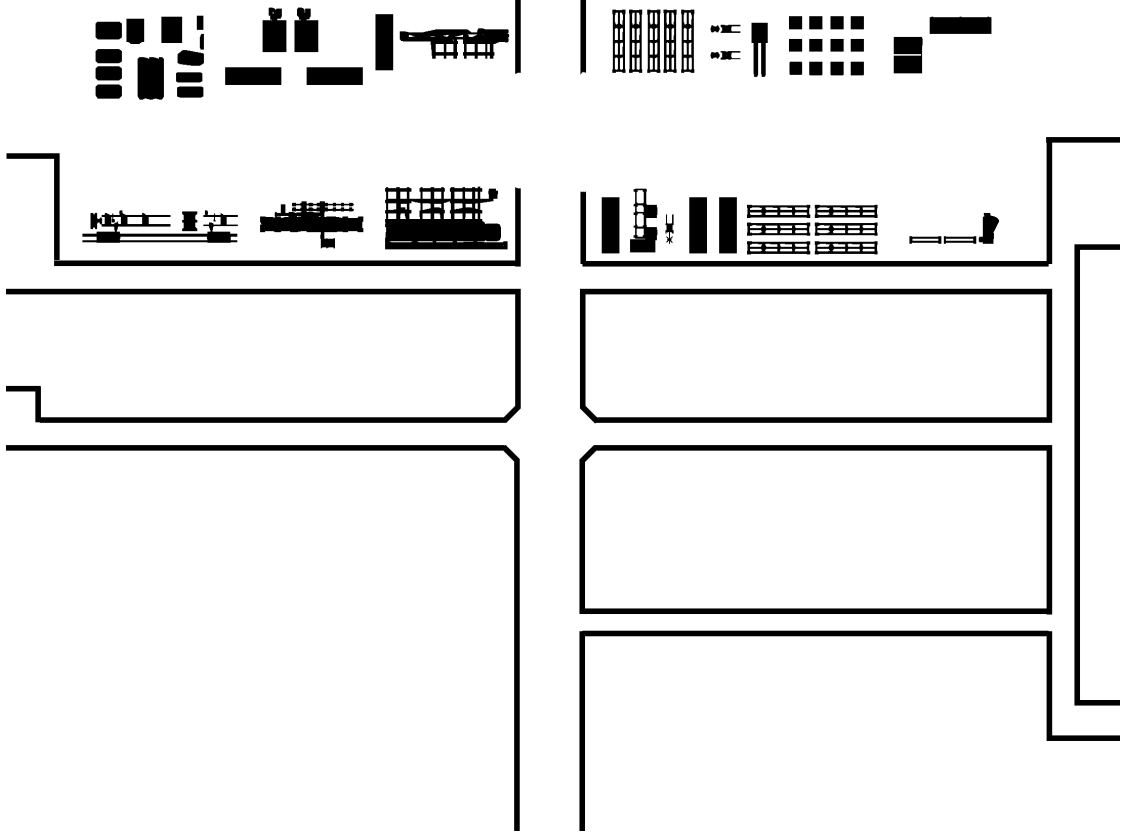


Figure 4.2: Top-down occupancy map of the Isaac factory line used for navigation and resource placement (robot layer).

metrics of `n10_i1_all_usable_unsup_best` are given in Table 4.2.

Table 4.2: Test metrics on the 134-episode usable pack. Primary metric: report P/R/F1 (station match and $|\hat{s} - s| \leq 3$ min).

Model	Rep. P	Rep. R	Rep. F1	Start MAE	Dur. MAE	Remain MAE	Who F1
XGBoost	—	—	—	—	—	—	—
LSTM	—	—	—	—	—	—	—
BTGCN	—	—	—	—	—	—	—
BSTAN	—	—	—	—	—	—	—
BNPDFormer	0.817	0.447	0.578	0.20	1.32	6.27	0.581

A correct *report* requires both the station identity and a start error of at most 3 min. Who-only F1 (0.581) nearly matches report F1 (0.578), so most who-correct predictions also satisfy the start tolerance; start MAE on matched events is 0.20 min and duration MAE is 1.32 min. Grid occupancy hot F1 is 0.613 (precision 0.779, recall 0.506) and is used for alignment, not for checkpointing. Remaining-order MAE is 6.27 windows (≈ 6.3 min). Per-type occupancy F1 is uneven: workbench 0.712, AGV 0.729, machine 0.676, gantry only 0.352. Gantry occupancy remains the weakest spatial class.

Ongoing events are recovered more readily than upcoming onsets (report recall 0.579 vs. 0.320

on 195 vs. 203 true events). This is the main failure mode for shop-floor warning. Cause-head accuracy is 0.935 against a majority-class baseline of 0.415 (macro recall 0.943 on transport delay, material shortage, starved upstream, and queue buildup). We still treat A.3 as a secondary analysis signal until a full confusion matrix is reported.

Tabular and graph-sequence baselines (XGBoost, LSTM, BTGCN/GCN+GRU [12], BSTAN/GAT+GRU [11]) are specified under the same pack, split, and report metric, but their training modules are not yet in `factory_bn/`; Table 4.2 therefore leaves those rows blank. Implementation patterns used throughout:

- **Process labels, not injection names.** Occupancy and events come from operational heat (turning point, queue, material starvation, inbound delay, upstream blockage, $\text{score} \geq 0.55$), not from copying L2 type strings. L2 enters X only as `disturbance_active_s`.
- **Episode-level splits.** `dataset.py` splits by episode, stratified by run; no window shuffling across train/val/test.
- **Precision-first reports.** A who-correct prediction still fails the primary metric if start error exceeds 3 min.
- **Streaming sidecar is for inference only.** `bn_agg --follow --closed-windows` can fill `derived/` while Isaac runs on GPU, but training always uses a full-episode aggregation without closed-window truncation.

4.4 Challenges

Despite the progress, several challenges remain in code and in the frozen numbers.

- **Label granularity.** Too fine a hot-mask explodes positive windows; too coarse merges distinct process phenomena. Aligning occupancy grids with sparse events (and not treating L2 shocks as events) required several revisions of `bn_agg`, because queue, process-state, and combined indicators do not coincide on the same windows [3, 7]. Current test hot positive rate is only 0.89%, so the task stays severely imbalanced.
- **Upcoming-event recall.** Ongoing congestion recall is 0.579; upcoming onset recall is 0.320. Raising future-start recall without collapsing the 0.817 report precision is the open modelling problem.
- **Gantry occupancy.** Gantry hot F1 (0.352) lags machines and workbenches. Dual-gantry interlocks and slow realistic motion make occupancy on transport assets harder to label and to predict.

- **Simulation completeness.** Even after watchdog and WIP-slot fixes, material and logistics runs still emit occasional `deadlock_reset` rows. Complete 10/10 episodes under disturbance remain expensive to collect.
- **Cause-head interpretation.** New-pack cause accuracy (0.935) beats the majority baseline (0.415), unlike the obsolete `evt` run whose `cause_acc=1.0` was spurious. A.3 is still not a headline claim until class-wise confusion is published.
- **Checkpoint drift.** Mixing `evt` ($N = 34$, paint-until-jobs-done) with `n10_i1_*` ($N = 38$, $H = 15$) invalidates every metric. History-12 checkpoints are likewise incompatible with `input_window=30`.
- **Missing baselines and product heads.** B2–B5 models are design-only in this tree. Product-side outputs B.1–B.3 are not wired; `job_kpi` currently only supplies remaining-job counts for the occupancy condition.

Chapter 5

Conclusion

This work presented a unified, verifiable pipeline for station-level bottleneck event forecasting from an Isaac Sim HRC factory twin [6]. The end-to-end system integrates (i) a process-faithful water-pipe environment with rule-based control (order 10, WIP 5), (ii) event-driven seven-table logging (`collector_version=v0.3`), (iii) single-axis $I = 1.0$ disturbance injection as environment context, (iv) offline window aggregation and unsupervised occupancy / event labels [5, 4], (v) a 38-node process–logistics graph with $F = 27$ features, (vi) a PDFormer-style spatiotemporal encoder (depth 4, width 64) [13], and (vii) precision-first event heads with remaining-time and cause auxiliaries.

On 134 usable episodes (split 92/17/25), BNPDFormer attains report precision/recall/F1 of 0.817/0.447/0.578, with start MAE 0.20 min and duration MAE 1.32 min. Remaining-order MAE is 6.27 min. These numbers show that precision-first station warnings with sub-minute start error on matched events are feasible in the twin; the open problem is upcoming-onset recall (0.320 vs. 0.579 for events already underway).

An obsolete evt checkpoint (hot F1 ≈ 0.165) must not be compared with this protocol. Gantry occupancy, mixed-dimension OOD, and fair baselines remain unfinished. Challenges—label granularity, imbalance, simulation completeness, cause-head interpretation, and checkpoint drift—were mitigated through occupancy–event alignment, episode filtering, precision-gated selection, and a frozen $H = 15$ evaluation protocol. The resulting stack balances twin controllability with event-centric supervision and is ready for the next phase: shared-split baselines, stronger upcoming-event recall, product-side heads, and live sidecar inference.

Chapter 6

Future Plans

Future work follows two tracks grounded in the current codebase. Track I focuses on research-grade evaluation and model refinement for station-level reports. Track II develops product-side forecasts and a deployable online toolkit.

6.1 Track I: Systematic Evaluation and Model Refinement

This track freezes the comparable protocol already used for Table 4.2 and raises upcoming-event quality. Priorities, in the order already used internally for collection and training scripts:

- Keep the short-order $I = 1.0$ four-dimension packs; do not mix $I = 2.0$ or 18-pipe norm20 into the $H = 15$ numbers. Re-aggregate after any `bn_agg` rule change, then retrain with `ckpt_metric=report_f1` and a precision floor.
- Implement B2–B5 in `factory_bn` (XGBoost on flattened windows, LSTM, GCN+GRU as BTGCN [12], GAT+GRU as BSTAN [11]) on the same `n10_i1_all_usable` split manifest, and fill the blank rows of Table 4.2.
- Evaluate `n10_mix_ood` (55 mixed-dimension episodes) under the same report metric as a true OOD test, without retraining on mixed labels.
- A.2 setting $\times$ station heatmaps by aggregating A.1 occupancy / events over machine, human, logistics, and material runs—no new network head.
- Publish the A.3 confusion matrix against `cause_majority_acc`; confirm that `material_shortage`, `transport_delay`, and `queue_buildup` remain separable. Improve gantry occupancy (type-balanced loss already exists in `FactoryBN.json`) before claiming transport-asset warnings.
- Turn on `use_stgnpp` only after occupancy precision is stable, so the sparse τ/d path is not clamped to trivial NLL [14].

6.2 Track II: Product-Side Forecasting and Shop-Floor Deployment

The second track extends beyond station reports toward production use [1, 6], using tables that already exist but are not yet supervision targets.

- Product-side heads B.1–B.3 from `job_kpi / job_trace`: on-time order flag, piece completion time, and subtask completion distributions. Today `job_kpi` only feeds remaining-job counts into the occupancy condition.
- Harden the streaming sidecar already documented in the follow-mode path: Isaac on GPU writes raw tables; CPU `bn_agg --follow` closes 60 s windows; `factory_bn.infer --device cpu` emits live reports without reading future labels. Do not train on follow-mode derived/.
- Package `batch_bn_collect.sh`, `python -m bn_agg`, `python -m factory_bn.export_dataset`, `python -m factory_bn.train`, and `python -m factory_bn.infer` as a reproducible toolkit with a data card (collector version, N , F , split seed, occupancy rule).

References

- [1] X. Xu, Y. Lu, B. Vogel-Heuser, and L. Wang. Industry 4.0 and Industry 5.0—inception, conception and perception. *Journal of Manufacturing Systems*, 61:530–535, 2021, doi: <https://doi.org/10.1016/j.jmsy.2021.10.006>.
- [2] Y. Fu, J. Chen, and W. Lu. Human-robot collaboration for modular construction manufacturing: Review of academic research. *Automation in Construction*, 158:105196, 2024, doi: <https://doi.org/10.1016/j.autcon.2023.105196>.
- [3] A. Skoogh, M. Thürer, M. Subramaniyan, A. Matta, and C. Roser. Throughput bottleneck detection in manufacturing: a systematic review of the literature on methods and operationalization modes. *Production & Manufacturing Research*, 11(1):2283031, 2023, doi: <https://doi.org/10.1080/21693277.2023.2283031>.
- [4] C. Roser, M. Nakano, and M. Tanaka. A practical bottleneck detection method. In *Proceeding of the 2001 Winter Simulation Conference*, vol. 2, pp. 949–953, 2001, doi: <https://doi.org/10.1109/WSC.2001.977398>.
- [5] L. Li, Q. Chang, and J. Ni. Data driven bottleneck detection of manufacturing systems. *International Journal of Production Research*, 47(18):5019–5036, 2009, doi: <https://doi.org/10.1080/00207540701881860>.
- [6] Y. Lu, C. Liu, K. I.-K. Wang, H. Huang, and X. Xu. Digital twin-driven smart manufacturing: Connotation, reference model, applications and research issues. *Robotics and Computer-Integrated Manufacturing*, 61:101837, 2020, doi: <https://doi.org/10.1016/j.rcim.2019.101837>.
- [7] C. Roser, M. Nakano, and M. Tanaka. Shifting bottleneck detection. In *Proceedings of the 2002 Winter Simulation Conference*, vol. 2, pp. 1079–1086, 2002.
- [8] X. Su, J. Lu, C. Chen, J. Yu, and W. Ji. Dynamic Bottleneck Identification of Manufacturing Resources in Complex Manufacturing System. *Applied Sciences*, 12(9):4195, 2022, doi: <https://doi.org/10.3390/app12094195>.
- [9] M. Subramaniyan, A. Skoogh, H. Salomonsson, P. Bangalore, and J. Bokrantz. A data-driven algorithm to predict throughput bottlenecks in a production system based on active periods of the machines. *Computers & Industrial Engineering*, 125:533–544, 2018, doi: <https://doi.org/10.1016/j.cie.2018.04.024>.
- [10] L. Ma, T. Qu, M. Thürer, Z. Wang, M. Yuan, and L. Liu. An integrated spatial-temporal neural network for proactive throughput bottleneck prediction in high-variety shops with

- complex job routings. *International Journal of Production Research*, 61(13):4437–4449, 2023, doi: <https://doi.org/10.1080/00207543.2022.2148769>.
- [11] X. Lai, T. Qiu, H. Shui, D. Ding, and J. Ni. Predicting future production system bottlenecks with a graph neural network approach. *Journal of Manufacturing Systems*, 67:201–212, 2023, doi: <https://doi.org/10.1016/j.jmsy.2023.01.010>.
- [12] J. Yin, R. Ma, and S. Ge. Predicting task bottlenecks in digital manufacturing enterprises based on spatio-temporal graph convolutional networks. *Frontiers of Engineering Management*, 12(4):736–753, 2025, doi: <https://doi.org/10.1007/s42524-025-4030-5>.
- [13] J. Jiang, C. Han, W. X. Zhao, and J. Wang. PDFormer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, pp. 4365–4373, 2023, doi: <https://doi.org/10.1609/aaai.v37i4.25556>.
- [14] G. Jin, L. Liu, F. Li, and J. Huang. Spatio-temporal graph neural point process for traffic congestion event prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, pp. 14268–14276, 2023, doi: <https://doi.org/10.1609/aaai.v37i12.26669>.